OpenAI Gym Code Report

Time-Aware Reinforcement Learning for Ball Catching under Visual Delay

ME5418 Group 30

1 Code Explanation

Each episode is one throw. A ball flies towards a net that slides in a vertical plane, and the agent controls the net's acceleration. The agent knows the net's own state at once, but it sees the ball only through camera frames that arrive late, at irregular times, and sometimes out of order. The code is a Python package, `ball_catch` (about 1,200 lines), plus a demo, a calibration script and 115 unit tests.

1.1 Organisation

File	What it does
`config.py`	All parameters in one dataclass, `EnvConfig`: timing, ball, net, camera, delay, reward.
`dynamics.py`	Ball path in closed form (gravity only); random throws; net motion with its limits.
`camera.py`	Pinhole camera: where the ball appears in the image and how big it looks, with pixel noise.
`delay.py`	Delay models (fixed, jitter, burst, replayed sequence) and the queue that delivers frames.
`env.py`	`BallCatchEnv`, a `gym.Env`: spaces, `reset`, `step`, reward and `info`.
`rendering.py`	Home-made Matplotlib visualisation (Figure 1).
`controllers.py`	Hand-written test policies: random, stationary, chase and a privileged oracle.
`evaluation.py`	Runs episodes and summarises success rate, return and delay statistics.
`demo.py`, `calibrate.py`	Demo script; pilot calibration (Table 1).
`tests/`	Unit tests, including Gym's own environment checker.

1.2 Main logic

`reset()` picks a flight time (0.6–1.2 s), a point where the ball will cross the net plane, and a launch point 6–9 m away, then solves for the launch velocity that joins them. The net starts at rest in the middle of its workspace.

`step(action)` does five things:

1. Scale the action (two numbers in $[-1, 1]$) to an acceleration; move the net for 1/60 s.
2. If the ball reaches the net plane in this step, stop at that exact moment and judge the catch.
3. Otherwise, take every camera frame that is due (30 Hz) and give each a random delivery delay.
4. Deliver the frames that have arrived. Keep the newest one only if it is newer than all before.
5. Build the observation and the reward.

Observation (11 numbers): the ball's image position and size in the latest accepted frame, a detection flag, a new-frame flag, the net's position and velocity, and two timing inputs: the *capture interval* between the last two accepted frames and the *observation age* of the latest one. `include_timing=False` removes the timing inputs; this gives the baselines without timestamps, on exactly the same throws. The true ball state is never observed; it is only in `info`, for evaluation and for the oracle.

Reward. Let e be the distance between the ball and net centres when the ball crosses the plane, and $R = 0.23$ m the largest e at which the ball still fits in the net:

$$r_{\text{end}} = \begin{cases} 1 + 0.5\,(1 - e/R) & e \le R \quad \text{(catch)} \\ -\min\big(1,\ (e - R)/1\,\text{m}\big) & e > R \quad \text{(miss)} \end{cases} \qquad r_{\text{step}} = -0.05\,\|a\|^2\,\Delta t.$$

How to run.

```
conda env create -f environment.yml && conda activate ball-catch
python demo.py                          # live window; --help lists policies and delay settings
python demo.py --save demo.gif          # write an animation instead
python -m unittest discover -s tests -t .
```

2 Existing Codes and Libraries

We did not use a robot simulator, and we did not copy code from online sources. The environment was written for this project with the help of AI coding assistants (see the declaration at the end).

- **Gym 0.26.2**: the `gym.Env` base class, `spaces.Box`, registration (`gym.make("BallCatch-v0")`) and the environment checker. Same version as the course's reference environment.
- **NumPy**: maths and random numbers; any episode can be replayed from its seed.
- **Matplotlib** for the visualisation; **Pillow** to save GIFs; Python's `heapq` (frame queue) and `unittest`.

Why no physics simulator? The ball follows a parabola we can compute exactly, and we do not simulate contact with the net, so a simulator such as PyBullet would add little. Writing it ourselves gives exact control over *when* each frame is captured and delivered, which is the core of our study. The price was building our own visualisation.

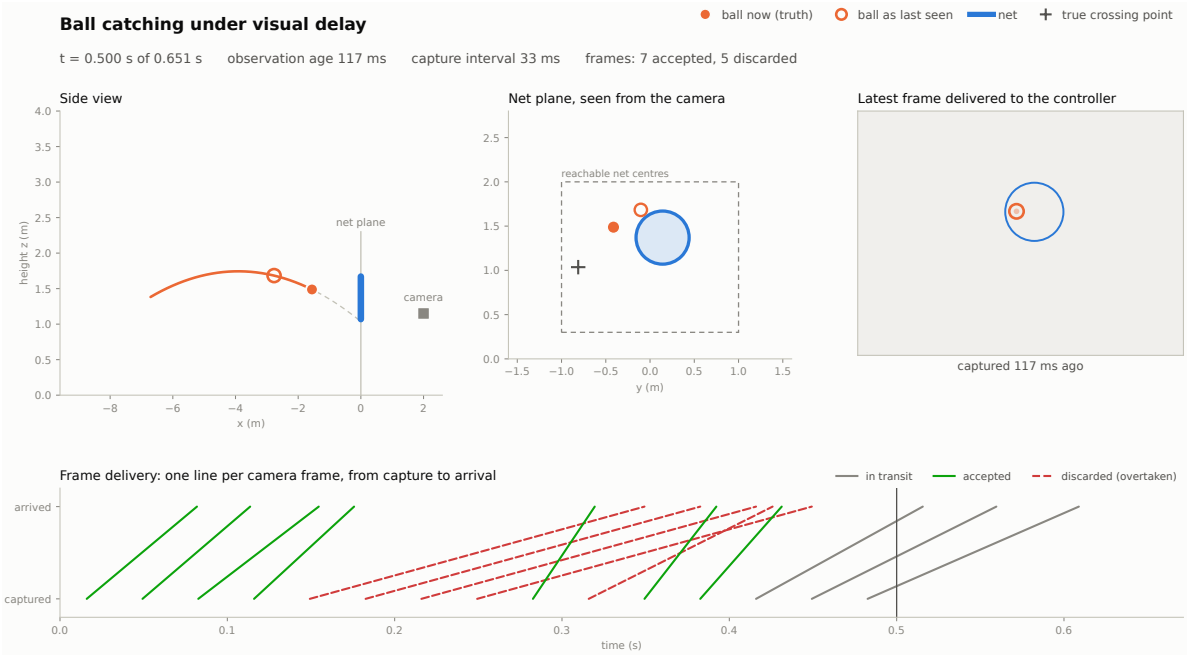


Figure 1: Visualisation, from `python demo.py --delay burst --seed 3` at $t = 0.5\,\text{s}$. Filled dot: the true ball. Hollow ring: where the controller last saw it. Bottom: one line per camera frame, from capture to arrival; red dashed frames were overtaken by newer ones and discarded. Here a stall of four frames leaves the controller with a 117 ms old view.

3 Reflections and Lessons Learned

Changes from the proposal.

- **Concrete timing (TA feedback).** Flight 0.6–1.2 s; camera 30 Hz (18–36 frames per throw); control 60 Hz; maximum delay 0.2 s, which is 17–33% of the flight.
- **Net limits from a pilot calibration (TA feedback).** Our first guess, $10\,\text{m/s}^2$, was too generous: an oracle starting 0.27 s late still caught 97% of balls, so delay hardly mattered. With $6\,\text{m/s}^2$ and 3 m/s, catching is feasible, not trivial, and sensitive to delay when there is little time to recover (Table 1).
- **Distance-based reward (TA feedback).** The proposal gave $+1$ or -1. Now a catch near the centre earns more, and a near miss costs less than a large miss. A catch still beats any miss by at least 1.
- **Simplifications.** Circular net opening. Image features come from a pinhole model with pixel noise, not from rendered images. The net always starts in the workspace centre.
- **Rules we pinned down.** If several frames arrive in one step, only the newest is accepted. Throws are built from a chosen flight time and crossing point, so difficulty is easy to control.

Table 1: Pilot calibration: success rate over the same 1,000 throws (`python calibrate.py`). *Chase* follows the line of sight to the last seen ball. *Oracle* knows the true crossing point but only starts moving after the delay plus two camera frames.

Fixed delay	Stationary net	Chase	Oracle, all throws	Oracle, flights $< 0.8\,\text{s}$
0	5.1%	18.8%	100.0%	100.0%
0.10 s	5.1%	10.8%	95.8%	86.6%
0.20 s	5.1%	8.0%	88.0%	62.0%

Lessons learned.

- **Check that delay matters before training.** Hand-written controllers showed this in minutes.
- **Delay also means lost frames.** With jitter of 80 ± 40 ms, about 36% of frames are overtaken and discarded, so the controller gets about 19 instead of 30 frames per second.
- **Reacting is not enough.** The chase policy catches only 19% even without delay: on high, slow throws the line of sight pulls the net up too early. This supports using memory and prediction.

Next steps. Re-check the calibration and tune the reward weights once RL policies train; implement the predictive baseline (timestamp-aware estimator plus PD control); run several environments in parallel for faster training.

AI tool declaration. We used Claude and GPT, through the AI assistant, to help design, write and test the environment code, and optimize the report. The project idea and all the requirements were proposed by us after discussion. We are responsible for the content and quality of the submitted work.